

Arquitectura medallón para un corpus de noticias de mercado

Bronze crudo, contrato de datos, carga idempotente y búsqueda semántica vectorial

Erika Jimenez

Práctica Módulo 1 — Diplomado de AI · Augmented Humans

1 de agosto de 2026

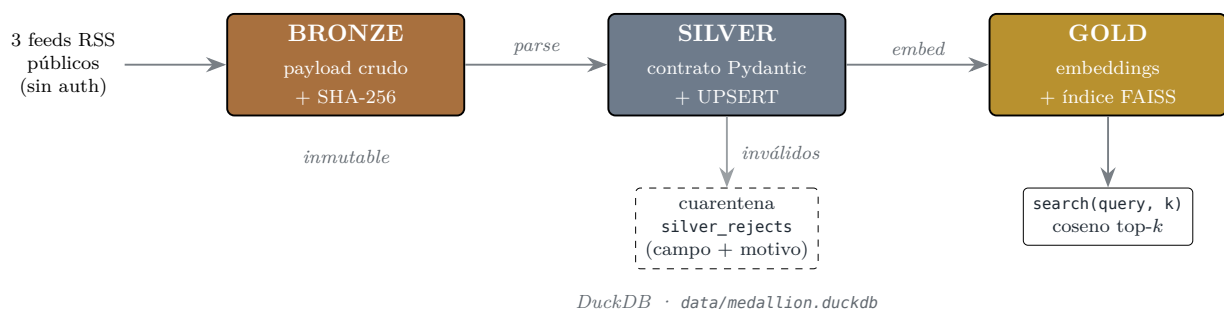
Resumen. Se implementó una arquitectura medallón completa sobre noticias de mercado cripto, ingeridas de tres feeds RSS públicos sin autenticación. La capa *Bronze* conserva dos lotes crudos íntegros con timestamp y huella SHA-256; *Silver* valida cada registro contra un contrato Pydantic v2 y desvía los inválidos a una tabla de cuarentena con el motivo del rechazo; la carga usa *staging* más *UPSERT* por clave natural, de modo que reprocesar un lote arroja `filas_nuevas = 0`; y *Gold* construye un índice vectorial FAISS de 384 dimensiones que expone una función de búsqueda semántica. Todo corre en contenedor vía `docker-compose` y fue ejecutado de punta a punta cuatro veces. Los cinco criterios de aceptación se cumplen con evidencia reproducible.

1. Objetivo y dominio

El requisito pedía una arquitectura medallón sobre un tema de interés propio, alimentada por una API o feed público real y sin autenticación. El dominio elegido son las **noticias de mercado de criptoactivos**, por continuidad con un proyecto propio de investigación cuantitativa: el corpus de noticias es texto libre, heterogéneo y sucio, que es justamente donde un contrato de datos y una búsqueda semántica aportan valor real y no decorativo.

Las fuentes son tres feeds RSS abiertos —CoinDesk, Cointelegraph y Decrypt—, sin API key, sin token y sin cuenta. El entregable es una base de código autocontenida (1 176 líneas de Python, 15 pruebas automatizadas) que no depende de ningún servicio de pago.

2. Arquitectura



El almacenamiento es **DuckDB** en archivo, montado como volumen del contenedor: el estado sobrevive entre corridas, condición necesaria para poder demostrar idempotencia. Las tablas son seis:

Capa	Tabla	Rol
Bronze	bronze_batches	payload crudo por (lote, fuente) + SHA-256
Silver	stg_news	<i>staging</i> , se recrea en cada corrida
Silver	silver_news	destino limpio, PK = clave natural
Silver	silver_rejects	cuarentena con campo, tipo y motivo
Gold	gold_news_embeddings	mapa news_id → posición en FAISS
—	load_audit	bitácora por corrida (evidencia de idempotencia)

3. Bronze: ingesta cruda

La regla de la capa es que el payload se guarde **tal como llegó**: sin parsear, sin limpiar HTML y sin recortar. Se persiste por partida doble — archivo en `data/medallion/bronze/<batch_id>/<fuente>.xml` y renglón en `bronze_batches`— con su huella SHA-256, de modo que cualquiera puede verificar después que el contenido no fue alterado. El `batch_id` es el timestamp UTC de la corrida, y la escritura usa `ON CONFLICT DO NOTHING`: un lote ya escrito nunca se sobrescribe.

Por qué no limpiar aquí. La tentación es normalizar el HTML en la ingesta. Si se hace, se pierde la capacidad de re-parsear el histórico con reglas nuevas cuando el contrato cambie —y con ella, el sentido de tener una capa Bronze. La limpieza vive en Silver.

Cuadro 1: Lotes crudos almacenados (salida de `bronze_batches`).

batch_id	fuentes	fetched_at	bytes crudos	SHA-256 distintos
20260801T001833Z	3	2026-08-01 00:18:33	116 368	3
20260801T001859Z	3	2026-08-01 00:18:59	116 368	3

4. Silver: contrato de datos y cuarentena

Cada registro se valida contra el modelo `NewsItem` (Pydantic v2) antes de tocar la tabla destino. El contrato es explícito y estricto: declara `extra="forbid"`, de modo que un campo inesperado en el feed es un rechazo y no un silencio.

```
class NewsItem(BaseModel):
    model_config = ConfigDict(extra="forbid", str_strip_whitespace=True)

    news_id: Annotated[str, Field(min_length=32, max_length=32)]
    source: Literal["coindesk", "cointelegraph", "decrypt"]
    title: Annotated[str, Field(min_length=10, max_length=300)]
    url: Annotated[str, Field(min_length=12, max_length=2000)]
    body: Annotated[str, Field(min_length=40, max_length=8000)]
    published_at: datetime
    symbols: list[str] = Field(default_factory=list)

    @field_validator("published_at")
    @classmethod
    def _fecha_plausible(cls, v: datetime) -> datetime:
        # ni del futuro ni de hace mas de tres anos
        ...

    @model_validator(mode="after")
    def _id_consistente(self) -> "NewsItem":
        if self.news_id != natural_key(self.source, self.url):
            raise ValueError("news_id no corresponde a sha256(source|url)")
```

```
return self
```

Listing 1: Contrato de datos (extracto de `contracts.py`).

Lo que no pasa el contrato no se descarta: se escribe en `silver_rejects` junto con el **campo**, el **tipo de error**, el **mensaje** y el **registro crudo completo** en JSON, de manera que el rechazo sea auditable y reprocesable. Cuando fallan varios campos a la vez se reporta la *causa raíz*: si la URL falta, `news_id` también falla por ser derivado, y el motivo relevante es el primero.

Cuadro 2: Validación por lote. Los cuatro rechazos (2 por lote) comparten motivo: campo `body`, tipo `string_too_short`, mensaje «*String should have at least 40 characters*» —notas cuyo resumen RSS venía vacío o de una sola línea—.

batch_id	leídas	válidas	rechazadas
20260801T001833Z	93	91	2
20260801T001859Z	93	91	2

5. Carga idempotente

La clave natural del dominio es `news_id = sha256(source | url)[:32]`, y es **PRIMARY KEY** de `silver_news`. Se eligió la URL y no el título —que cambia con las ediciones del editor— ni el GUID del feed —que cada fuente formatea distinto—.

El flujo es *staging* y luego **MERGE**. La tabla `stg_news` se recrea en cada corrida y se deduplica internamente, porque un mismo feed puede repetir una nota dentro del mismo lote; sin ese paso el **UPSERT** intentaría actualizar dos veces la misma fila.

```
CREATE OR REPLACE TABLE stg_news AS
SELECT * FROM stg_news
QUALIFY row_number() OVER (PARTITION BY news_id ORDER BY published_at DESC) = 1;

INSERT INTO silver_news (news_id, source, title, url, body, published_at,
                        symbols, content_hash, first_batch_id, last_batch_id,
                        inserted_at, updated_at)
SELECT news_id, source, title, url, body, published_at, symbols,
       content_hash, batch_id, batch_id, ?, ?
FROM stg_news
ON CONFLICT (news_id) DO UPDATE SET
    title      = excluded.title,
    body       = excluded.body,
    published_at = excluded.published_at,
    content_hash = excluded.content_hash,
    last_batch_id = excluded.last_batch_id,
    updated_at  = CASE
        WHEN silver_news.content_hash IS DISTINCT FROM excluded.content_hash
        THEN excluded.updated_at
        ELSE silver_news.updated_at
    END;
```

Listing 2: Deduplicación en staging y **MERGE** por clave natural (`silver.py`).

El **CASE** sobre `content_hash` —resumen de título, cuerpo, fecha y símbolos— distingue dos situaciones que suelen confundirse: «*ya había visto esta nota*» y «*esta nota cambió*». Un reproceso puro deja `updated_at` intacto; una corrección editorial en la fuente sí queda registrada. Cada corrida escribe su renglón en `load_audit`.

Las corridas 3 y 4 son el caso estricto que pide el criterio: reprocesar el *mismo* `batch_id` ya cargado. Ambas dan `filas_nuevas = 0`. La corrida 2 es un caso adicional e interesante: bajó un

Cuadro 3: Bitácora `load_audit`: cuatro ejecuciones de punta a punta.

<code>run_id</code>	<code>batch_id</code>	válidas	<code>filas_nuevas</code>	<code>filas_actualizadas</code>
<code>run-20260801T001834Z</code>	<code>...T001833Z</code>	91	91	0
<code>run-20260801T001900Z</code>	<code>...T001859Z</code>	91	0	0
<code>run-20260801T001939Z</code>	<code>...T001833Z</code>	91	0	0
<code>run-20260801T001953Z</code>	<code>...T001859Z</code>	91	0	0

lote *nuevo* (timestamp y SHA-256 propios) veintiséis segundos después del primero; como los feeds no habían publicado nada en ese lapso, el MERGE reconoció las 91 notas por clave natural y tampoco insertó nada. La deduplicación es por contenido, no por lote.

6. Ausencia de duplicados

```
SELECT news_id, count(*) FROM silver_news GROUP BY news_id HAVING count(*) > 1;
-- (0 filas)
-- filas en silver_news = 91 · news_id distintos = 91
```

Listing 3: Consulta de verificación y su resultado.

La cuarentena también es idempotente: su llave primaria es (`batch_id`, `source`, `record_ord`), así que reprocesar un lote no infla la tabla de rechazos —un detalle que se escapa con facilidad y que está cubierto por una prueba automatizada.

7. Gold: índice vectorial y búsqueda semántica

El campo relevante es el texto de la nota: se embebe `título`. `cuerpo` con `sentence-transformers/all-MiniLM-L6` (384 dimensiones, CPU, local) y se indexa en un `faiss.IndexFlatIP` sobre vectores normalizados en L2, de modo que el producto interno es la similitud coseno. El índice se persiste en disco y la construcción es incremental: sólo se embeben los `news_id` que aún no están en `gold_news_embeddings`, cuya columna `vec_ord` guarda la posición en el índice.

Se eligió el índice exhaustivo y no uno aproximado (IVF, HNSW) porque a esta escala —decenas o centenas de documentos— la búsqueda exacta es instantánea y un índice aproximado sólo agregaría hiperparámetros que calibrar sin ganancia medible.

```
def search(query: str, k: int = 5) -> list[dict]:
    """Devuelve las k notas mas cercanas al texto libre."""
    idx = _load_index()
    qv = embed([query]) # normalizado en L2
    scores, ords = idx.search(qv, min(k, idx.ntotal))
    # vec_ord -> JOIN con silver_news -> filas con score coseno
```

Listing 4: Función de búsqueda semántica expuesta (`gold.py`).

Cuadro 4: Estado del índice tras las cuatro corridas.

vectores	<code>vec_ord</code> distintos	modelo	dim
91	91	<code>all-MiniLM-L6-v2</code>	384

Cuadro 5: Tres consultas en lenguaje natural y sus tres primeros resultados.

score	titular recuperado
<i>«lawsuit and regulatory crackdown by financial authorities»</i>	
0.4627	New York sues Kalshi over alleged illegal gambling operation
0.4091	New York sues Kalshi, alleges it offers a gambling platform «plain and...»
0.3604	The good and the bad of perps, according to crypto traders
<i>«institutional money flowing into spot ETFs»</i>	
0.5666	Bitcoin ETFs post \$233M inflows , pushing week back into the green
0.5391	Bitcoin, Ethereum Wobble as Fed Holds Rates Steady
0.4162	Aviva Investors launches tokenized fund after Central Bank of Ireland...
<i>«sharp market selloff and price volatility»</i>	
0.4124	Bitcoin's calm is back and so is the setup for a volatility explosion
0.3029	Bitcoin, ether fall, equities rally with broader crypto market on track...
0.3014	Bitcoin holds monthly gain, faces «choppy» August as «forced-selling»...

Consultas demostradas

Por qué es búsqueda semántica y no textual. La consulta *«lawsuit and regulatory crackdown by financial authorities»* recupera *«New York sues Kalshi...»*: ninguna de las palabras de la consulta aparece en ese titular —un LIKE '%lawsuit%' habría devuelto cero filas—. Lo mismo ocurre con *«institutional money flowing into spot ETFs»*, que encuentra *«inflows»*, y con *«sharp market selloff»*, que encuentra *«volatility explosion»*. La recuperación opera sobre el significado del texto, no sobre sus caracteres.

8. Contenedorización y reproducción

La imagen pesa 459 MB: se instala explícitamente la build *CPU* de PyTorch desde su índice, porque la build por defecto arrastra CUDA (unos 2,5 GB) que aquí no se usa. El contenedor corre como UID 1000 para que lo escrito en el volumen pertenezca al usuario del anfitrión y no a *root*.

```
cd medallion
docker compose build

docker compose run --rm medallion                # corrida 1: lote nuevo
docker compose run --rm medallion                # corrida 2: segundo lote
docker compose run --rm medallion python -m medallion run --batch-id <ID>
docker compose run --rm medallion python -m medallion search "texto libre"
```

Listing 5: Reproducción completa desde cero.

El directorio *data/* se monta como volumen: *bronze* crudo, base DuckDB e índice FAISS sobreviven al contenedor, que es precisamente lo que permite demostrar idempotencia entre ejecuciones separadas.

9. Pruebas automatizadas

Quince pruebas corren sin red y sin descargar el modelo —los embeddings se sustituyen por un hash determinista y el lote de *Bronze* se siembra con un RSS sintético—:

- aceptación del contrato y rechazo campo por campo (URL inválida, cuerpo corto, título

corto, fuente desconocida, fecha futura, símbolo fuera de la whitelist, campo extra, `news_id` inconsistente);

- cuarentena que registra el motivo correcto;
- triple reproceso del mismo lote con `filas_nuevas = 0`;
- dos lotes de contenido idéntico sin generar duplicados;
- cuarentena que no se infla al reprocesar;
- índice FAISS incremental y orden decreciente de *score* en la búsqueda.

```
$ pytest medallion/tests -o addopts=""  
15 passed in 1.21s
```

10. Cumplimiento de los criterios de aceptación

Criterio	Mínimo exigido	Obtenido
Bronze	2+ lotes crudos, timestamp, sin transformar	2 lotes de 116 368 bytes, 3 payloads con SHA-256 cada uno, guardados byte a byte en disco y en base
Contrato	validación explícita, cuarentena con motivo	<code>NewItem</code> (Pydantic v2, <code>extra="forbid"</code>); 93 leídas → 91 válidas y 2 en cuarentena por lote, con campo, tipo, mensaje y registro crudo
Idempotencia	reproceso da <code>filas_nuevas = 0</code>	staging + <code>ON CONFLICT DO UPDATE</code> ; 3 de 4 corridas con <code>filas_nuevas = 0</code> , incluidos dos reprocesos del mismo <code>batch_id</code>
Duplicados	<code>COUNT(*) > 1</code> por clave natural = 0 filas	0 filas; 91 registros con 91 <code>news_id</code> distintos
Gold	índice vectorial funcional, 1+ consulta semántica	FAISS <code>IndexFlatIP</code> de 384-d con 91 vectores; 3 consultas demostradas

11. Decisiones de diseño y limitaciones

Tres decisiones merecen mención explícita. Primera, la clave natural es la URL y no el título ni el GUID, por las razones ya expuestas. Segunda, Bronze es inmutable por diseño (`ON CONFLICT DO NOTHING`), lo que permite re-parsear el histórico si el contrato evoluciona. Tercera, el `content_hash` separa la re-observación del cambio real, que es lo que hace que la métrica `filas_actualizadas` signifique algo.

Las limitaciones se enuncian sin adorno:

- el modelo `all-MiniLM-L6-v2` es **monolingüe en inglés**, igual que el corpus; una consulta en español recupera resultados notablemente peores, y una versión bilingüe exigiría cambiar a `paraphrase-multilingual-MiniLM-L12-v2`;
- los feeds RSS entregan sólo el resumen y no la nota completa, de modo que los embeddings describen el *abstract*, no el cuerpo entero;
- la inferencia de símbolos es por palabras clave y no por reconocimiento de entidades: alcanza para etiquetar, no para desambiguar.

A. Anexo: salida literal de la ejecución

Lo que sigue es la salida sin editar de `docker compose run --rm medallion python -m medallion evidence`, ejecutada tras las cuatro corridas.

```
=====
EVIDENCIA – arquitectura medallón · 2026-08-01 00:20:12Z · db=/app/data/medallion.duckdb
=====

1) BRONZE – lotes crudos almacenados sin transformar
  batch_id      fuentes  fetched_at      bytes_crudos  sha256_distintos
  -----
  20260801T001833Z  3      2026-08-01 00:18:33.296624  116368      3
  20260801T001859Z  3      2026-08-01 00:18:59.808019  116368      3

2) CONTRATO Pydantic – válidos vs cuarentena por lote
  batch_id      leídas  válidas  rechazadas
  -----
  20260801T001833Z  93      91      2
  20260801T001859Z  93      91      2

  CUARENTENA – motivos de rechazo (silver_rejects)
  campo  tipo_error      motivo
  -----
  body   string_too_short  String should have at least 40 characters  4

3) IDEMPOTENCIA – staging + MERGE por clave natural (news_id)
  run_id      batch_id      executed_at      válidas  filas_nuevas  filas_actualizadas
  -----
  run-20260801T001834Z  20260801T001833Z  2026-08-01 00:18:34.389178  91      91      0
  run-20260801T001900Z  20260801T001859Z  2026-08-01 00:19:00.572338  91      0      0
  run-20260801T001939Z  20260801T001833Z  2026-08-01 00:19:39.373690  91      0      0
  run-20260801T001953Z  20260801T001859Z  2026-08-01 00:19:53.335896  91      0      0

4) DUPLICADOS – news_id con COUNT(*) > 1 (debe ser 0 filas)
  (0 filas)
  filas en silver_news = 91 · news_id distintos = 91

5) GOLD – índice vectorial FAISS
  vectores  vec_ord_distintos  modelo      dim
  -----
  91      91      sentence-transformers/all-MiniLM-L6-v2  384

  BÚSQUEDA SEMÁNTICA – "lawsuit and regulatory crackdown by financial authorities"
  score  fuente      título
  -----
  0.4627  cointelegraph  New York sues Kalshi over alleged illegal gambling operation  ["ETH"]
  0.4091  coindesk      New York sues Kalshi, alleges it offers a gambling platform 'plain a  []
  0.3604  coindesk      The good and the bad of perps, according to crypto traders  []

  BÚSQUEDA SEMÁNTICA – "institutional money flowing into spot ETFs"
  score  fuente      título
  -----
  0.5666  cointelegraph  Bitcoin ETFs post $233M inflows, pushing week back into the green  ["BTC"]
  0.5391  decrypt      Bitcoin, Ethereum Wobble as Fed Holds Rates Steady  ["BTC", "ETH"]
  0.4162  cointelegraph  Aviva Investors launches tokenized fund after Central Bank of Irelan  ["XRP"]

  BÚSQUEDA SEMÁNTICA – "sharp market selloff and price volatility"
  score  fuente      título
  -----
  0.4124  coindesk      Bitcoin's calm is back and so is the setup for a volatility explosio  ["BTC"]
  0.3029  coindesk      Bitcoin, ether fall, equities rally with broader crypto market on tr  ["BTC", "ETH"]
  0.3014  coindesk      Bitcoin holds monthly gain, faces 'choppy' August as 'forced-selling  ["BTC"]

=====
[PASA] bronze_2_lotes
[PASA] cuarentena_con_motivo
[PASA] reproceso_filas_nuevas_0
[PASA] sin_duplicados
[PASA] indice_vectorial
=====
```